

CSE-3203 Final Examination 2024

Complete Questions and Answers

University of Dhaka - Design and Analysis of Algorithms-II

How to use these notes

The questions are transcribed from the two provided exam-page screenshots. The answers are written in exam-ready form, with calculations, pseudocode, proofs, and diagrams where useful. For one slightly unclear phrase in Q7(b)(i), the solution states the standard interpretation explicitly.

Question 1(a) - 4 marks

Define **primary clustering** in the context of hashing. Explain the effect of primary clustering and the mechanism to reduce it.

Answer 1(a)

Primary clustering occurs in open addressing, especially with **linear probing**, when consecutive occupied locations form long contiguous blocks (clusters).

With linear probing,

$$h(k, i) = (h'(k) + i) \bmod m.$$

If a new key hashes anywhere inside a cluster, it is placed at the first free slot after the cluster. This makes the cluster still larger.

Effects:

- successful and unsuccessful searches require more probes;
- insertion becomes slower as the cluster grows;
- performance deteriorates rapidly when the load factor α becomes large.

Ways to reduce it:

- use **quadratic probing**, which spreads probes nonlinearly and removes primary clustering;
- use **double hashing**, which gives different probe steps to different keys and usually spreads keys even better;
- keep the load factor moderate and resize/rehash when the table becomes too full.

Important distinction

Quadratic probing removes *primary* clustering but keys with the same initial hash can still follow the same sequence (secondary clustering). Double hashing also reduces this effect.

Question 1(b) - 6 marks

Consider inserting the keys

10, 22, 31, 4, 15, 28, 17, 88, 59

into a hash table of length $m = 11$ using open addressing with auxiliary hash function

$$h'(k) = k \bmod 11.$$

Illustrate the result of inserting the keys using:

- quadratic probing with $C_1 = 1$ and $C_2 = 3$, and
- double hashing with $h_1(k) = k \bmod 11$ and $h_2(k) = 1 + (k \bmod (m - 1))$.

Answer 1(b)**(i) Quadratic probing**

$$h(k, i) = (h'(k) + i + 3i^2) \bmod 11, \quad i = 0, 1, 2, \dots$$

Key	Probe positions tried	Final slot
10	10	10
22	0	0
31	9	9
4	4	4
15	4, 8	8
28	6	6
17	6, 10, 9, 3	3
88	0, 4, 3, 8, 8, 3, 4, 0, 2	2
59	4, 8, 7	7

Final table:

index	0	1	2	3	4	5	6	7	8	9	10
key	22	—	88	17	4	—	28	59	15	31	10

(ii) Double hashing

$$h(k, i) = (h_1(k) + i h_2(k)) \bmod 11,$$

where

$$h_2(k) = 1 + (k \bmod 10).$$

Key	$h_2(k)$	Probe positions tried	Final slot
10	1	10	10
22	3	0	0
31	2	9	9
4	5	4	4
15	6	4, 10, 5	5
28	9	6	6
17	8	6, 3	3
88	9	0, 9, 7	7
59	10	4, 3, 2	2

Final table:

index	0	1	2	3	4	5	6	7	8	9	10
key	22	—	59	17	4	15	28	88	—	31	10

Question 1(c) - 4 marks

Prove that the expected number of probes in an unsuccessful search is at most

$$\frac{1}{1 - \alpha}$$

for an open-address hash table with load factor $\alpha = n/m < 1$, assuming uniform hashing.

Answer 1(c)

Let X be the number of probes made by an unsuccessful search. The search needs more than i probes only if the first i probed positions are all occupied.

Under uniform hashing,

$$\Pr(X > i) = \frac{n}{m} \frac{n-1}{m-1} \cdots \frac{n-i+1}{m-i+1} \leq \left(\frac{n}{m}\right)^i = \alpha^i.$$

For a positive integer-valued random variable,

$$E[X] = \sum_{i \geq 0} \Pr(X > i).$$

Hence

$$E[X] \leq \sum_{i \geq 0} \alpha^i = \frac{1}{1-\alpha},$$

because $0 \leq \alpha < 1$.

Therefore,

$$E[X] \leq \frac{1}{1-\alpha}.$$

Question 2(a) - 7 marks

Let P be a finite set of points, and let A be the area of the convex hull of P . Consider points $\{p_1, p_2, \dots, p_n\} \subset P$ and define a new point

$$q = \lambda_1 p_1 + \lambda_2 p_2 + \cdots + \lambda_n p_n, \quad 0 < \lambda_i < 1.$$

Let $P' = P \cup \{q\}$, and let B be the area of the convex hull of P' . Determine the relative order between A and B .

Answer 2(a)

Since

$$P \subseteq P',$$

we necessarily have

$$\text{conv}(P) \subseteq \text{conv}(P').$$

Adding a point can never make the convex hull smaller. Therefore its area cannot decrease:

$$A \leq B.$$

If the coefficients additionally satisfy the usual convex-combination condition

$$\sum_{i=1}^n \lambda_i = 1,$$

then $q \in \text{conv}(P)$, so the hull does not change at all and

$$A = B.$$

Without that additional condition, q may lie outside the old hull, in which case $B > A$; but $A > B$ is impossible.

Question 2(b) - 7 marks

Given two axis-aligned ellipses

$$E_1 : \frac{(x-h_1)^2}{a_1^2} + \frac{(y-k_1)^2}{b_1^2} = 1, \quad E_2 : \frac{(x-h_2)^2}{a_2^2} + \frac{(y-k_2)^2}{b_2^2} = 1,$$

formulate an efficient algorithm to determine whether the two ellipses intersect.

Answer 2(b)

A robust constant-dimensional method is to parameterize the first ellipse and test whether the second ellipse's implicit equation becomes zero.

Parameterize E_1 by

$$x(t) = h_1 + a_1 \cos t, \quad y(t) = k_1 + b_1 \sin t, \quad 0 \leq t < 2\pi.$$

Substitute this point into E_2 and define

$$F(t) = \frac{(h_1 + a_1 \cos t - h_2)^2}{a_2^2} + \frac{(k_1 + b_1 \sin t - k_2)^2}{b_2^2} - 1.$$

The ellipse boundaries intersect exactly when

$$F(t) = 0 \text{ for some } t \in [0, 2\pi).$$

Efficient procedure:

1. First reject quickly using bounding boxes. If

$$[h_1 - a_1, h_1 + a_1] \cap [h_2 - a_2, h_2 + a_2] = \emptyset$$

or the corresponding y -intervals are disjoint, return FALSE.

2. Otherwise find the minimum and maximum of $F(t)$ on $[0, 2\pi]$. Candidate extrema occur at endpoints and where $F'(t) = 0$.
3. Because F is continuous, the two boundaries intersect/touch iff

$$\min F \leq 0 \leq \max F.$$

Using the substitution $u = \tan(t/2)$, the equation $F(t) = 0$ becomes a fixed-degree polynomial (at most quartic), so this can be solved in constant algebraic dimension.

If “ellipse” means the filled region

If boundary curves do not cross, also test whether the center of one ellipse lies inside the other. Containment then counts as intersection of the filled regions.

Question 3(a) - 7 marks

Consider the text

$$T = \text{AABAACAADAABAABA}$$

and the pattern

$$P = \text{AABA}.$$

Find all occurrences of P in T using the Knuth-Morris-Pratt (KMP) algorithm. List all character comparisons made between the text and pattern during matching, and mention the total number of comparisons.

Answer 3(a)

First compute the prefix function of $P = \text{AABA}$:

$$\pi = [0, 1, 0, 1].$$

The matches occur at 0-based shifts

$$[0, 9, 12],$$

or at 1-based text positions

$$[1, 10, 13].$$

Using logical character comparisons (each text-pattern equality test counted once), the trace is:

#	Text pos.	$T[i]$	Pattern pos.	$P[j]$	Result
1	1	A	1	A	match
2	2	A	2	A	match
3	3	B	3	B	match
4	4	A	4	A	match (occurrence 1)
5	5	A	2	A	match
6	6	C	3	B	mismatch
7	6	C	2	A	mismatch
8	6	C	1	A	mismatch
9	7	A	1	A	match
10	8	A	2	A	match
11	9	D	3	B	mismatch
12	9	D	2	A	mismatch
13	9	D	1	A	mismatch
14	10	A	1	A	match
15	11	A	2	A	match
16	12	B	3	B	match
17	13	A	4	A	match (occurrence 2)
18	14	A	2	A	match
19	15	B	3	B	match
20	16	A	4	A	match (occurrence 3)

Thus the total number of logical text-pattern comparisons is

$$[20].$$

Why KMP saves work

After a full match, $\pi[4] = 1$, so KMP keeps one matched A instead of restarting from zero. After a mismatch it follows prefix links, so the text index never moves backward.

Question 3(b) - 7 marks

You are given a two-dimensional text grid of characters with R rows and C columns, and a two-dimensional pattern grid with r rows and c columns, where $r \leq R$ and $c \leq C$. The pattern occurs if some $r \times c$ subgrid matches it character by character.

Explain how KMP can be adapted to efficiently find all occurrences of the pattern grid within the text grid. Mention the time complexity.

Answer 3(b)

The 2-D problem can be handled in two stages: **horizontal row matching** followed by **vertical KMP**.

Preprocessing. Treat each pattern row

$$P_0, P_1, \dots, P_{r-1}$$

as a row-string of length c , and assign an identifier to each distinct pattern row. Preprocess these row strings so that, while scanning a text row, we can report which pattern-row identifier (if any) matches

at every starting column.

Horizontal stage. For every text row, use KMP-style prefix processing to locate occurrences of the pattern-row strings. At every text position (i, j) , record a row identifier saying which pattern row matches the length- c substring beginning at column j .

Vertical stage. For each possible starting column j , the recorded identifiers down the rows form a one-dimensional sequence. The desired 2-D pattern corresponds to the sequence

$$(0, 1, \dots, r-1)$$

(or the corresponding pattern-row IDs). Run KMP vertically on this sequence. Whenever all r IDs match consecutively, an occurrence of the $r \times c$ pattern has been found.

With row matching organized so that all row patterns are processed together, the total work is linear in the input size:

$$O(RC + rc).$$

The memory requirement is $O(RC)$ if all row-match IDs are stored, or $O(C + r)$ with streaming column states.

Straightforward version

If one runs an independent KMP scan for every one of the r pattern rows against every text row, the bound becomes $O(rRC)$. The efficient two-stage implementation avoids that repeated work by sharing the horizontal row-matching stage.

Question 4(a) - 7 marks

Assume a partially filled two-dimensional array where empty cells are denoted by 0. Design an efficient backtracking algorithm to complete the grid such that:

- each row contains the digits 1 to 9 exactly once;
- each column contains the digits 1 to 9 exactly once;
- each 3×3 subgrid contains the digits 1 to 9 exactly once.

Answer 4(a)

This is the standard **Sudoku backtracking** problem.

At every step, choose an empty cell (r, c) . Try each digit $d \in \{1, \dots, 9\}$ that is absent from:

- row r ,
- column c ,
- the 3×3 box containing (r, c) .

If a choice later leads to a dead end, undo it and try the next digit.

SOLVE-SUDOKU(G):

 if no empty cell remains:
 return TRUE

(r, c) = choose an empty cell
 preferably one with minimum candidates (MRV)

 for $d = 1$ to 9:
 if SAFE(G, r, c, d):
 $G[r][c] = d$
 if SOLVE-SUDOKU(G):

```

        return TRUE
        G[r][c] = 0          // backtrack

    return FALSE

SAFE(G,r,c,d):
    return d not in row r
        and d not in column c
        and d not in the 3x3 box of (r,c)

```

Using row/column/box bitmasks makes each safety test $O(1)$. The worst case is exponential, commonly bounded by $O(9^E)$ for E empty cells, but constraints and MRV pruning reduce the practical search dramatically.

Question 4(b) - 7 marks

There are n day laborers who work on assignments in varying groups, and the individual earnings of each laborer for every assignment are given. For any subteam S , define its **joint-earning** as the total earnings obtained from assignments in which all members of S worked together. Given a threshold k , list all subteams whose joint-earning is greater than k . Propose an efficient branch-and-bound algorithm.

Answer 4(b)

Represent a state by a partial team S and a set R of workers that may still be added.

Let

$$C(S) = \{a : \text{every worker in } S \text{ participated in assignment } a\}.$$

The exact joint earning of S is

$$J(S) = \sum_{a \in C(S)} \sum_{i \in S} e_{i,a},$$

where $e_{i,a} \geq 0$ is worker i 's earning on assignment a .

For any descendant $S' \supseteq S$, the common assignment set can only shrink:

$$C(S') \subseteq C(S).$$

A safe optimistic upper bound is therefore

$$UB(S, R) = \sum_{a \in C(S)} \sum_{i \in S \cup (R \cap W_a)} e_{i,a},$$

where W_a is the set of workers participating in assignment a . This assumes every remaining worker whose earning could help can be added without losing any common assignment, so it can only overestimate a descendant's earning.

```

ENUMERATE(S, R, C):
    J = exactJointEarning(S, C)
    if S is nonempty and J > k:
        output S

    UB = optimisticBound(S, R, C)
    if UB <= k:
        return          // prune entire subtree

    if R is empty:
        return

    choose next worker x from R

```

```
// include x
C2 = C intersect assignmentsContaining(x)
ENUMERATE(S union {x}, R after x, C2)

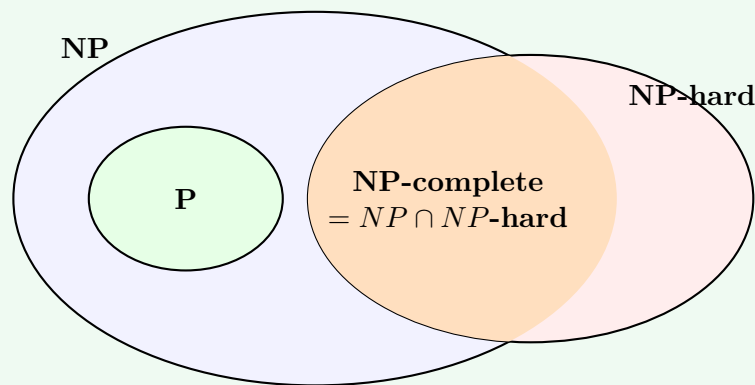
// exclude x
ENUMERATE(S, R after x, C)
```

Efficiency. Store each worker's assignment set as a bitset. Then computing $C(S \cup \{x\})$ is a fast bitwise AND. Branch-and-bound can eliminate huge families of teams whenever $UB \leq k$. Worst-case enumeration remains exponential because the output itself can contain exponentially many subteams.

Question 5(a) - 6 marks

Draw a Venn diagram that correctly represents the relationships between P , NP , NP -hard, and NP -complete. Using appropriate definitions, explain the relationships among these four classes.

Answer 5(a)



Definitions and relationships:

- P : decision problems solvable in polynomial time by a deterministic algorithm.
- NP : decision problems whose YES certificates can be verified in polynomial time.
- NP -hard: a problem H is NP -hard if every problem in NP can be polynomial-time reduced to H . An NP -hard problem need not itself belong to NP .
- NP -complete: problems that are both in NP and NP -hard:

$$NP\text{-complete} = NP \cap NP\text{-hard}.$$

Also,

$$P \subseteq NP.$$

Whether $P = NP$ is unknown; the usual diagram draws P as a proper subset only to illustrate the widely believed case $P \neq NP$.

Question 5(b) - 8 marks

Prove that the following feature-selection problem is NP -complete.

You are given a set of features and a collection of labeled examples. Each example is described by the features present in it and has binary label 0 or 1. Two examples are **conflicting** if they have different labels. A selected feature set is **consistent** if every conflicting pair can be distinguished using at least one selected feature. Given a positive integer k , determine whether there exists a subset of at most k features that distinguishes all conflicting pairs.

Answer 5(b)

We prove NP-completeness by reduction from **SET COVER**.

1. The problem is in NP. A certificate is a set F' of at most k selected features. For every pair of examples with different labels, check whether at least one selected feature has different values on the two examples. This takes polynomial time. Hence the problem is in NP.

2. SET COVER $\leq_p$ Feature Selection. Take an instance of SET COVER:

$$U = \{u_1, \dots, u_m\}, \quad \mathcal{S} = \{S_1, \dots, S_n\}, \quad k.$$

Create one feature f_j for each set S_j .

Create one special example x_0 with label 0 and all feature values 0. For each universe element u_i , create an example x_i with label 1, defined by

$$x_i[f_j] = \begin{cases} 1, & u_i \in S_j, \\ 0, & u_i \notin S_j. \end{cases}$$

The only relevant conflicting pairs are (x_0, x_i) , since all x_i 's have the same label 1.

A selected feature f_j distinguishes x_0 from x_i exactly when

$$u_i \in S_j.$$

Therefore a set of at most k features distinguishes every conflicting pair iff the corresponding k sets cover every element of U .

The construction is polynomial. Thus SET COVER reduces to the feature-selection problem. Since SET COVER is NP-complete, the feature-selection problem is NP-hard; combined with membership in NP,

Feature Selection is NP-complete.

Question 6(a) - 6 marks

Given a unique character set of size 100 and a maximum pattern length of 7, determine the minimum number of bits required in an integer to design the Rabin-Karp algorithm with linear runtime complexity.

Answer 6(a)

To eliminate hash collisions completely, encode a pattern of at most 7 characters as an exact base-100 integer.

The number of possible length-7 strings is

$$100^7 = 10^{14}.$$

So we need enough bits to represent values up to $100^7 - 1$:

$$b = \lceil \log_2(100^7) \rceil = \lceil 7 \log_2 100 \rceil.$$

Since

$$7 \log_2 100 \approx 46.506,$$

we get

$$\boxed{b = 47 \text{ bits}}.$$

With exact rolling fingerprints there are no spurious hash matches, so the scan remains linear in the text length after preprocessing.

Question 6(b) - 4 marks

Given text

$$T = \text{ABABABABC}$$

and pattern

$$P = \text{ABABC},$$

find all occurrences of P in T using Rabin-Karp. Use

$$h(S) = \left(\sum_{k=0}^{m-1} \text{value}(s_k) d^{m-1-k} \right) \bmod q,$$

with $d = 4$, $q = 13$, and character values $A = 0, B = 1, C = 2$.

Answer 6(b)

Pattern length is $m = 5$.

Pattern hash:

$$\begin{aligned} h(\text{ABABC}) &= (0 \cdot 4^4 + 1 \cdot 4^3 + 0 \cdot 4^2 + 1 \cdot 4 + 2) \bmod 13 \\ &= (64 + 4 + 2) \bmod 13 \\ &= 70 \bmod 13 = \boxed{5}. \end{aligned}$$

The five length-5 windows of T are:

Shift (0-based)	Window	Hash mod 13	Match?
0	ABABA	3	No
1	BABAB	0	No
2	ABABA	3	No
3	BABAB	0	No
4	ABABC	5	Yes

Only the window at shift 4 has the same hash; direct character verification confirms the match. Therefore

$$\boxed{P \text{ occurs at 0-based position } 4}$$

or at 1-based position

$$\boxed{5}.$$

The rolling update can use

$$\begin{aligned} h_0 &= d^{m-1} \bmod q = 4^4 \bmod 13 = 9, \\ t_{s+1} &= (d(t_s - \text{value}(T[s])h_0) + \text{value}(T[s+m])) \bmod q. \end{aligned}$$

Question 6(c) - 4 marks

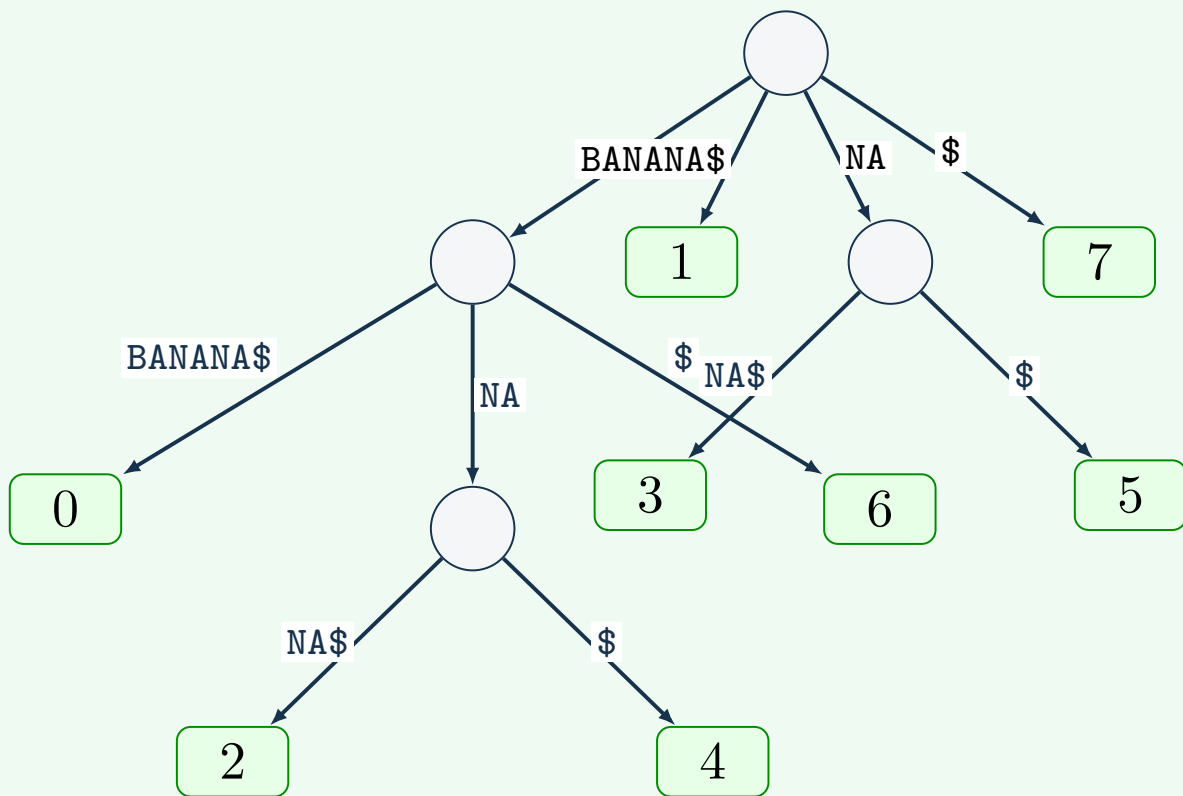
Draw the suffix tree for the string **ABANANA**.

Answer 6(c)

Append a unique terminal marker:

$$X = \text{ABANANA\$}.$$

The suffixes are indexed 0 through 7.



The root-to-leaf strings are exactly the suffixes:

0	ABANANA\$	4	ANA\$
1	BANANA\$	5	NA\$
2	ANANA\$	6	A\$
3	NANA\$	7	\$

Question 7(a) - 4 marks

The ski-rental problem is defined as follows. You are going skiing for an unknown number of days. Renting skis costs 500 Taka per day and buying skis costs 1500 Taka. Every day you must decide whether to continue renting for one more day or buy a pair of skis.

Mention the features of an online algorithm. Define the competitive ratio using a strategy for this ski-rental problem.

Answer 7(a)

Features of an online algorithm:

- input arrives over time, one request/event at a time;
- the algorithm must decide using only the information seen so far;
- future requests are unknown;
- decisions are normally irrevocable or costly to undo;
- performance is measured against an optimal offline algorithm that knows the entire future.

For a minimization problem, A is c -competitive if there is a constant β such that

$$A(\sigma) \leq c \text{OPT}(\sigma) + \beta$$

for every request sequence σ .

Ski strategy. Since

$$\frac{1500}{500} = 3,$$

rent for the first two days; if skiing continues to day 3, buy at the beginning of day 3.
For N skiing days,

$$A(N) = \begin{cases} 500, & N = 1, \\ 1000, & N = 2, \\ 2500, & N \geq 3, \end{cases}$$

while

$$\text{OPT}(N) = \min(500N, 1500).$$

Hence the worst ratio is

$$\max_N \frac{A(N)}{\text{OPT}(N)} = \frac{2500}{1500} = \boxed{\frac{5}{3}}.$$

So this threshold strategy is $5/3$ -competitive for these particular costs.

Question 7(b) - 10 marks

Consider the online linear-list search problem. The cost of accessing an item is proportional to its distance from the head of a linked list, and a request sequence of accesses is given. The objective is to reorder the list so that total access cost is minimized.

- (i) (4 marks) Consider a case in which the accessed item is moved two positions forward, and the request sequence is only for the same item of the list. Calculate the competitive ratio.
- (ii) (6 marks) Prove that no algorithm has a competitive ratio better than 2 for the linear-list search problem.

Answer 7(b)

(i) Move-ahead-two strategy.

The scan is slightly unclear after the words “the item of the list.” We use the natural standard interpretation: the request sequence repeatedly requests one fixed item, and for the worst case that item starts at the last position n .

The online strategy moves the accessed item forward by only two positions after each access. Thus its access positions are

$$n, n-2, n-4, \dots$$

until it reaches the front.

If $n = 2r$, consider the first r requests. The online cost is

$$A = r(r+1)$$

because

$$2r + (2r-2) + \dots + 2 = r(r+1).$$

The offline optimum pays $2r$ on the first request, moves the item to the front, and then pays 1 thereafter:

$$\text{OPT} = 2r + (r-1) = 3r-1.$$

Therefore

$$\frac{A}{\text{OPT}} = \frac{r(r+1)}{3r-1} = \Theta(r) = \Theta(n).$$

So the move-ahead-two rule is **not constant-competitive**; its competitive ratio can grow linearly with the list size:

$$\boxed{\text{competitive ratio} = \Theta(n)}.$$

(ii) Lower bound of 2 for deterministic online list search.

Consider any deterministic online algorithm A . An adaptive adversary always requests the item currently at the last position of A 's list. Therefore A pays n for every request. For m requests,

$$A(\sigma) = mn.$$

Now compare with a fixed offline ordering. If a list ordering is chosen uniformly at random, every requested item has expected position

$$\frac{n+1}{2}.$$

Hence the expected total access cost of that random fixed ordering is

$$\frac{m(n+1)}{2}.$$

By the averaging principle, at least one fixed ordering has cost no larger than this. Therefore the offline optimum satisfies, up to an initial rearrangement additive constant,

$$\text{OPT}(\sigma) \leq \frac{m(n+1)}{2} + O(n^2).$$

For long sequences,

$$\frac{A(\sigma)}{\text{OPT}(\sigma)} \geq \frac{mn}{m(n+1)/2 + O(n^2)} \rightarrow \frac{2n}{n+1}.$$

As $n \rightarrow \infty$,

$$\frac{2n}{n+1} \rightarrow 2.$$

Thus no deterministic online algorithm can have competitive ratio strictly below 2 in the asymptotic worst case:

$$\boxed{c \geq 2.}$$

Last-Minute Revision Sheet

Hashing

- Primary clustering: long contiguous occupied runs, especially in linear probing.
- Quadratic: $h(k, i) = (h'(k) + C_1i + C_2i^2) \bmod m$.
- Double hash: $h(k, i) = (h_1(k) + ih_2(k)) \bmod m$.
- Uniform hashing unsuccessful search: $E[X] \leq 1/(1 - \alpha)$.

Geometry and Backtracking

- Adding a point to a set cannot shrink its convex hull: $A \leq B$.
- Ellipse boundary intersection: parameterize one ellipse and solve $F(t) = 0$.
- Sudoku: choose empty cell, try safe digits, recurse, undo on failure.
- Branch-and-bound: compute a safe upper bound; if $UB \leq k$, prune.

String Matching

- KMP pattern AABA: $\pi = [0, 1, 0, 1]$; matches in the given text at 0-based shifts 0, 9, 12.
- 2-D KMP: horizontal row matching + vertical KMP on row IDs.
- Rabin-Karp exact base-100 code for length 7 needs 47 bits.
- ABABC occurs once in ABABABABC, at 0-based shift 4.

Complexity and Online Algorithms

- $P \subseteq NP$, NP -complete = $NP \cap NP$ -hard.
- Feature selection is NP-complete via SET COVER.
- Ski rental with rent 500, buy 1500: buy on day 3 after two rentals; ratio 5/3.
- Deterministic online list search has lower bound 2.